COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:		**DATE SUBMITTED:**	
NAME:	Adrian Walter D. Santos	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 Stacks

1. PROCEDURE OUTPUT

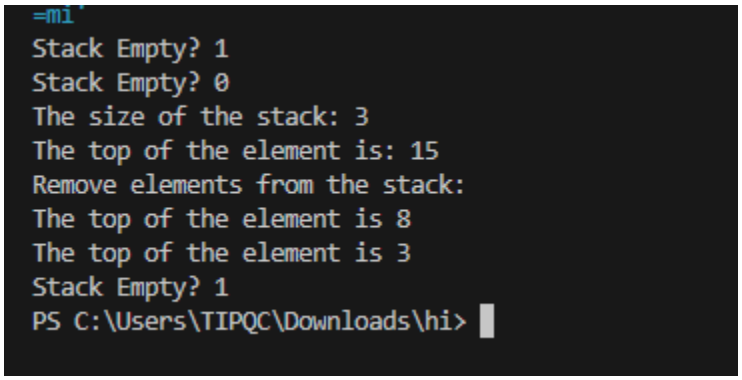
Code

```cpp
#include <iostream>
#include <stack>


int main(){
    std::stack<int> newStack;
    std::cout << "Stack Empty? "<<newStack.empty() << std::endl;
    //push stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the element is: "<< newStack.top()<<std::endl;
    std::cout<<"Remove elements from the stack: \n";
    newStack.pop();
    std::cout<<"The top of the element is "<< newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the element is "<< newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;

    return 0;
}
```

Output	
This ILO taught us how to push elements, pop elements, show the top of the slack, and show if the stack is empty or not. This activity helped us gain a solid grasp on the basics of stack handling, by learning and familiarizing these concepts we are now able to apply this into implementations such as a linked list.	

Code

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData, choice;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();

int main(){
    std::cout<< "Enter the number of max elements for the new stack: ";
    std::cin>>i;
    while(true){
        std::cout<<"Stack Operations"<<std::endl;
        std::cout<<"1. PUSH 2.POP 3.TOP 4.EMPTY"<<std::endl;
        std::cin>>choice;
        switch (choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<<std::endl;
            break;
            default: std::cout<<"Invalid Choice"<<std::endl;
            break;
        }
    }
    return 0;
};

```

Main function and function declaration

```
bool isEmpty(){
    //Create a condition to check whether the top element is equal to -1
    if(top == -1) return true;
    return false;

}

bool isFull(){
    //check if we reached the maximum capactiy of the array
    if(top < i-1) return true;
    return false;
}
void push(){
    //check if full -> if yes return an error
    if(top > 100){
        std::cout<<"stack overflow";
        return;
    }

    std::cout << "New value to add to the stack"<<std::endl;
    std::cin >> newData;
    //increment the top(index by 1) then assing the new data;
    stack[++top] = newData;
}

void Top(){
    //check if stack is empty
    if(isEmpty()){
        std::cout<<"Stack underflow."<<std::endl;
    }

    std::cout<<"The element on the top of the stack is" <<stack[top]<<std::endl;
}

void pop(){
    //check if the stack is empty
    if(isEmpty()){
        std::cout<<"Stack underflow."<<std::endl;
    }

    //display the top value
    std::cout<<"popping: "<<stack[top];
    //decrement the top value from the stack
    top--;
}
```

Functions

```cpp
int main(){
    std::cout<< "Enter the number of max elements for the new stack: ";
    std::cin>>i;
    while(true){
        std::cout<<"Stack Operations"<<std::endl;
        std::cout<<"1. PUSH 2.POP 3.TOP 4.EMPTY 5.DISPLAY"<<std::endl;
        std::cin>>choice;
        switch (choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty()<<std::endl;
            break;
            case 5: displayStack();
            break;
            default: std::cout<<"Invalid Choice"<<std::endl;
            break;
        }
    }
    return 0;
};
```

```cpp
void displayStack(){
    //Check if stack is empty
    if(isEmpty){
        std::cout<<"Cannot be an empty stack"<<std::endl;
    }

    //
    for(int i = top; i >= 0; i--){
        std::cout<<stack[i]<<',';
    }
    std::cout<<std::endl;


}
```

Output

```
Enter the number of max elements for the new stack: 3
Stack Operations
1. PUSH 2.POP 3.TOP 4.EMPTY 5.DISPLAY
1
New value to add to the stack
23
Stack Operations
1. PUSH 2.POP 3.TOP 4.EMPTY 5.DISPLAY
1
New value to add to the stack
21
Stack Operations
1. PUSH 2.POP 3.TOP 4.EMPTY 5.DISPLAY
1
New value to add to the stack
42
Stack Operations
1. PUSH 2.POP 3.TOP 4.EMPTY 5.DISPLAY
5
Cannot be an empty stack
42,21,23,
Stack Operations
1. PUSH 2.POP 3.TOP 4.EMPTY 5.DISPLAY
```

This ILO taught us how to apply the keywords into functions. This helped us create a stack through user input which gave us multiple options that we can go through such as push, pop, top, empty, and display. This further honed our knowledge when it came to stack handling.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Array

Code

```cpp
#include <iostream>
#include <string>

class ArrayStack {
private:
    char arr[100];
    int top;

public:
    ArrayStack() { top = -1; }

    bool isEmpty() {
        return top == -1;
    }

    void push(char c) {
        if (top < 99) {
            arr[++top] = c;
        }
    }

    char pop() {
        if (!isEmpty()) {
            return arr[top--];
        }
        return '\0';
    }
};
```

```
bool MatchPair(char open, char close) {
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}

void BalanceSym(std::string expr) {
    ArrayStack stack;
    bool isBalanced = true;
    int i = 0;
    int length = expr.length();


    while (i < length) {
        char ch = expr[i];


        if (ch == '(' || ch == '{' || ch == '[') {
            stack.push(ch);
        }

        else if (ch == ')' || ch == '}' || ch == ']') {

            if (stack.isEmpty()) {
                isBalanced = false;
                break;
            }

            char topChar = stack.pop();
            if (!MatchPair(topChar, ch)) {
                isBalanced = false;
                break;
            }
        }
```

```
        i++;
    }

    if (!stack.isEmpty()) {
        isBalanced = false;
    }

    if (isBalanced) {
        std::cout << "Expression is balanced / result is valid.\n";
    } else {
        std::cout << "Expression is unbalanced / result is invalid\n";

}
}

int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B)+[C-D]}"
    };

    for (const std::string& expr : expressions) {
        std::cout << "Expression: " << expr << std::endl;
        std::cout << "Output: ";
        BalanceSym(expr);
        std::cout << "-----------------------------------" << std::endl;
    }

    return 0;
}
```

Output

```
Expression: (A+B)+(C-D)
Output: Expression is balanced / result is valid.
-----------------------------------
Expression: ((A+B)+(C-D)
Output: Expression is unbalanced / result is invalid
-----------------------------------
Expression: ((A+B)+[C-D])
Output: Expression is balanced / result is valid.
-----------------------------------
Expression: ((A+B)+[C-D]}
Output: Expression is unbalanced / result is invalid
-----------------------------------
PS C:\Users\apple\Downloads\Stacks (HOA4)\ILOC>
```

LINKED LIST

Code

```cpp
#include <iostream>
#include <string>
#include "stackList.h"

bool MatchPair(char open, char close) {
    if (open == '(' && close == ')') return true;
    if (open == '{' && close == '}') return true;
    if (open == '[' && close == ']') return true;
    return false;
}

void BalanceSym(std::string expr) {
    bool isBalanced = true;
    int i = 0;
    int length = expr.length();

    while (i < length) {
        char ch = expr[i];

        if (ch == '(' || ch == '{' || ch == '[') {
            push<char>(ch);
        }
        else if (ch == ')' || ch == '}' || ch == ']') {
            if (isEmpty<char>()) {
                isBalanced = false;
                break;
            }

            char topChar = pop<char>();
            if (!MatchPair(topChar, ch)) {
                isBalanced = false;
                break;
            }
        }

        i++;
    }
```

```
    if (!isEmpty<char>()) {
        isBalanced = false;

        while (!isEmpty<char>()) {
            pop<char>();
        }
    }

    if (isBalanced) {
        std::cout << "Expression is balanced / result is valid.\n";
    } else {
        std::cout << "Expression is unbalanced / result is invalid\n";
    }
}

int main() {
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B)+[C-D]}"
    };

    for (const std::string& expr : expressions) {
        std::cout << "Expression: " << expr << std::endl;
        std::cout << "Output: ";
        BalanceSym(expr);
        std::cout << "-----------------------------------" << std::endl;
    }

    return 0;
}
```

Main file

```cpp
#ifndef STACK_LIST_H
#define STACKL_LIST_H

//class node that contains data and the next
template <typename T>
class Node{
    public:
    T data;  //VALUE THAT WE NEED TO STORE
    Node* next; //POINTER TO THE NEXT NODE
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

// operations for the stack

// push
template <typename T>
void push(T newData){
    // Create a mew mpde and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data= newData;

    //new node points to current top, then becomees the new top
    newNode->next = head<T>;
    head<T> = newNode;

}
// pop
template <typename T>
T pop(){
    //check if the newData
    if(head<T> == nullptr){
        std::cout<< "Stack overflow: "<<std::endl;
    }
```

```
    // save for the current top node
    Node<T>* temp = head<T>;

    // store the data
    T temVal = temp->data;

    // move the head to the next node
    head<T> = head<T>->next;

    // free the removed node
    delete temp;

    //return the removed value
    return temVal;

}

// top
template<typename T>
void top(){
    // error catching
    if(head<T> == nullptr){
        std::cout<<"Error the stack is empty";
        return;
    }
    //display the top node
    std::cout<<"Top of stack: "<<head<T>->data <<std::endl;
}
```

```
// isEmpty
template<typename T>
bool isEmpty(){
    return head <T> == nullptr;
}

// Display all
template <typename T>
void displayAll(){
    if(isEmpty<T>()){
        std::cout<<"Stack is empty!\n";
        return;
    }

    std::cout << "Stack(top to bottom): ";
    Node<T>* current = head<T>;
    while (current != nullptr){
        std::cout<<current->data;
        if (current->next != nullptr) std::cout << " ->";
        current = current->next;
    }
    std::cout<<std::endl;
}

#endif
```

Header file

Output

```
Expression: (A+B)+(C-D)
Output: Expression is balanced / result is valid.
-----------------------------------
Expression: ((A+B)+(C-D)
Output: Expression is unbalanced / result is invalid
-----------------------------------
Expression: ((A+B)+[C-D])
Output: Expression is balanced / result is valid.
-----------------------------------
Expression: ((A+B)+[C-D]}
Output: Expression is unbalanced / result is invalid
-----------------------------------
PS C:\Users\apple\Downloads\Stacks (HOA4)\ILOC>
```

Expression	Valid? [Y/N]	Output	Analysis
(A+B)+(C-D)	Y	Expression: (A+B)+(C-D) Output: Expression is balanced / result is valid.	The symbols present in the expression don't violate any rules which makes it balanced.
(A+B)+(C-D)	N	Expression: ((A+B)+(C-D) Output: Expression is unbalanced / result is invalid	The expression has a parenthesis that does not have a match on the other side, which makes it unbalanced
((A+B)+[C-D])	Y	Expression: ((A+B)+[C-D]) Output: Expression is balanced / result is valid.	All of the symbols in the expression have a match which is why it resulted in a valid
((A+B]+[C-D]}	N	Expression: ((A+B]+[C-D]} Output: Expression is unbalanced / result is invalid	The covering has a mismatch between a parenthesis and a curly bracket which makes it invalid

Conclusion

Stack's principle being FILO allows it to be applied into different kinds of implementation. Its uses can vary and overall it can help optimize the code and make it easier to run. In this activity we learned how to use its function in

our code in finding if an expression is balanced or not. The activity also tasked us to do it in both array and linked list implementation, this will help us easily integrate stacks in both of these implementations as we now have the experience to do it.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.